

Small Language Model Optimization Experiment Report

Junchao Zeng 3036748103

Final scaled model test BPB: 1.765 (a 15.9% reduction compared to the baseline of 2.10)
Validation BPB of the improved model with original dimensions: 1.784 (1200 training steps; significant improvement under the same model size)

AI Usage Description: Proposing ideas for code modification directions and writing the specific code; detailing the specific content and significance of modification approaches; analyzing results and creating tables.

1. Project Overview

The goal of this project is to optimize the official baseline GPT model, and minimize BPB on the WikiText-2 dataset without exceeding resource constraints. A lower BPB indicates stronger text modeling and compression capabilities of the model.

Hard Constraints

- Inference CPU time must not exceed 5 times that of the baseline
- Peak memory during evaluation must not exceed 4 GiB
- Uncompressed model file size must not exceed 64 MiB
- Only the provided training data may be used; external pre-trained weights are prohibited. Hyperparameter tuning is conducted only on the validation set, and the test set is run only once for the final evaluation.

Baseline Specification

The official baseline is a small GPT model with 4 layers and 128 dimensions, totaling approximately 1.088 million parameters. After 1200 training steps, its test-set BPB is around 2.10.

2. Improvements Implemented

We made modifications from two directions: model architecture and training strategy, and then conducted a round of moderate model scaling. The official dataset, evaluation scripts and testing logic remained unchanged throughout the process.

2.1 Architectural Modifications

(1) Replace LayerNorm with RMSNorm The mean offset term in normalization is removed, retaining only root-mean-square normalization. It features fewer parameters

and faster computation, while also stabilizing training. For small models, it achieves performance comparable to LayerNorm at a lower cost.

(2) Replace GELU with SwiGLU gated activation The original GELU is a fixed activation function. SwiGLU adds an additional gated path, using a sigmoid gate to control information flow, resulting in stronger nonlinear representational capacity. We adopted the variant with $3\times$ hidden dimension, which significantly improves the model's fitting capacity with only a modest increase in parameters.

(3) Add QK Normalization Before computing attention, Query and Key are each normalized separately. This avoids the problem of unstable attention scores in small models, and makes the dependency relations learned by the attention mechanism more robust.

(4) Replace absolute positional embeddings with RoPE (Rotary Position Embedding) Instead of trainable position vectors, positional information is injected into Q and K through rotation. The advantage is that it can better model the relative positions between tokens, has stronger generalization ability in short contexts, and adds no trainable parameters at all. We adjusted the parameters for the 256-token short context window to achieve finer positional encoding.

(5) Deep residual scaled initialization Following the practice of GPT-2, the initial weights of the projection layers in the residual branch are scaled down by the square root of the number of layers. This alleviates the problem of gradient vanishing during training of deep models, making training more stable and convergence more complete.

(6) Add Dropout Dropout is added to the residual paths of both attention output and MLP output. During training, a portion of neurons are randomly deactivated, forcing the model to learn more general patterns and reducing overfitting.

2.2 Training Strategy Modifications

(1) Warmup + cosine annealing learning rate The learning rate increases linearly from 0 at the beginning to avoid abrupt gradients right after initialization; afterwards, the learning rate decreases gradually following a cosine curve, with finer adjustments in later stages. This achieves much better convergence quality than a fixed learning rate.

(2) EMA weight moving average A smoothed set of model weights is maintained during training, and this averaged weight is used for final evaluation. It smooths out weight fluctuations during training, leading to better generalization ability with no increase in inference cost at all.

(3) Weight decay + gradient clipping AdamW optimizer with weight decay is used for L2 regularization to prevent overfitting; meanwhile, the maximum norm of gradients is limited to prevent gradient explosion and stabilize training.

2.3 Two Model Versions

We developed two versions of the model, corresponding to different performance and cost tradeoffs:

- Original-size improved version: 4 layers and 128 dimensions, same as the baseline. Only architecture and training strategies are modified, with very little parameter increase.
- Final scaled version: 5 layers and 160 dimensions, with moderately enlarged model size for further performance improvement.

3. Experimental Result Comparison

All models use a fixed `seed=17` and are trained for 1200 steps, with the exact same total number of training tokens as the baseline for fair comparison.

3.1 Original-size Improved Model (Same Size as Baseline)

With the exact same number of layers and dimensions as the baseline, relying solely on architecture and training optimizations, the validation set results are as follows:

- Validation set BPB: 1.784
 - Single validation runtime: 4.77 seconds
 - Total training time for 1200 steps: 305.5 seconds
 - Parameter count: approximately 1.297 million, 19% more than the baseline
- Compared with the baseline validation BPB of around 2.1, the BPB drops by about 0.31 under the same model size, with a very significant improvement, indicating high returns from architecture and training improvements.

3.2 Full Performance Comparison

表格

Model	Dataset	BPB	Improvement vs. Baseline	Parameter Count
Official Baseline	Test Set	2.10	-	1.088M
Architecture + Training Optimization (Original Size)	Validation Set	1.784	~15%	1.297M
Final Scaled Version	Test Set	1.765	15.9%	2.451M

It can be seen that: Relying solely on architecture and training strategy optimization can bring very large performance improvements at the same model size, and even the inference speed is close to the baseline. With moderate scaling, the test-set BPB drops directly to 1.765, a reduction of nearly 16%.

4. Ablation Study: Contribution of Each Improvement

We conducted control-variable ablation experiments on the original-size improved model. Only one improvement is removed while all other conditions remain unchanged, and the increase in validation set BPB reflects the individual contribution of that improvement.

表格

Ablation Item	Validation Set BPB	Increase vs. Full Model
Full original-size improved model	1.784	-
Remove SwiGLU, revert to GELU	1.844	+0.06
Remove RoPE, revert to absolute position embedding	1.834	+0.05
Remove RMSNorm, revert to LayerNorm	1.804	+0.02

Ablation Conclusions

SwiGLU has the largest effect. Gated activation is indeed the core method to improve performance for small models, and can alone bring a 0.06 BPB reduction. RoPE ranks second. Relative position encoding is more suitable for short text modeling than absolute embeddings, and has zero parameter cost with extremely high cost-effectiveness.

5. Analysis and Discussion

Why These Methods Work

The biggest problem with small models is "insufficient representational capacity". SwiGLU, RoPE and other methods boost the model's nonlinear capacity and sequence modeling ability at very low cost; RMSNorm, QK normalization, deep initialization and other methods help the model train more stably and fully realize its inherent potential.

Training techniques essentially "help the model converge better". With few training steps and small model size, training is prone to insufficient convergence or oscillation. The combination of warmup, EMA and weight decay exactly solves these problems.

The high return from scaling is because the constraints of 64 MiB and $5\times$ inference time are actually very lenient, and the baseline model is far from the limits. Therefore, moderately increasing the model size improves performance much faster than it increases cost.

Performance and Cost Tradeoff

We divide the improvements into three tiers, in descending order of priority:

- Zero-cost improvements: RoPE, RMSNorm, EMA, learning rate scheduling. No additional parameters, no additional inference latency, mandatory.
- Low-cost improvements: SwiGLU, QK normalization. Slight increase in parameters and computation, but large performance gains, well worth it.
- Model scaling: Delivers the most performance improvement, with linear cost growth, but still far from the limits. Suitable for pursuing higher ranking.

Limitations

Improvements are only stacked one by one, and no systematic search for the optimal parameter combination is conducted, so there may be better combinations.

6. Conclusion

In this optimization, we started from three directions: architecture, training and model size. Under strict compliance with resource constraints, we achieved certain results: At the same model size, relying solely on architecture and training optimization reduces the validation set BPB from around 2.1 to 1.784, a huge improvement. After moderate scaling, the final test-set BPB reaches 1.765, a 15.9% reduction compared to the baseline.

The core takeaways are: Gated activation and rotary position encoding are the two most effective architectural improvements for boosting small model performance. Training techniques such as EMA and cosine annealing have zero inference cost and stable returns, and must be included. When there is a large resource margin, the model size can be moderately increased.